TA7: An Introduction to Stan

30/10/2026

```
# You can check out the useful Stan forum if you have any Stan-related inquiries (https://discourse.mc-
# For doubts about how to install Stan, see https://mc-stan.org/install/

# Install the cmdstanr R package
# Run this only once:
# install.packages(
#   "cmdstanr",
#   repos = c("https://stan-dev.r-universe.dev",
#             getOption("repos"))
# )

# Load the package
library(cmdstanr)

# Check that the C++ toolchain is ready
# On Windows, this checks for a suitable Rtools setup.
# check_cmdstan_toolchain()

# Install CmdStan (only needed once)
# install_cmdstan()

# Verify the installation
# cmdstan_path()
# cmdstan_version()


# Create folder to store .stan files
if (!dir.exists("./stan")) {
  dir.create("./stan")
}

library(posterior)
library(bayesplot)
library(ggplot2)
```

Stan is a probabilistic programming language

It implements state of the art HMC algorithms.

Specifically, its default is the No-U-Turn Sampler by Hoffman and Gelman (2014) with some further improvements.

Given a target probability density function, the sampling is divided in

1. **Adaptation**: the algorithm learns the "optimal" stepsize ε, integration time L and mass matrix M of the momentum conditional distribution. The idea is to optimize these parameters so that the acceptance rate of the algorithm is 80%. This is done by first reaching the typical set, then learning M and finally L and ε.
2. **Burnin**: A number of iterations that will be discarded
3. Proper **sampling**

By default 1000 iterations of adaptation, 0 of burnin and 1000 of sampling (no thinning!)

```
# Define the Stan model string
# data section -> input, what is given to the model
# parameters section -> the unknown quantities that will be sampled by the MCMC
# model section -> specify the actual model you are using

normal_code <- "
    data {
        int<lower=0> dim;
        matrix[dim, dim] cov_chol;
    }

    parameters {
        vector[dim] x;
    }

    model {
        vector[dim] mu = rep_vector(0, dim);
        x ~ multi_normal_cholesky(mu, cov_chol);
    }
"

# Set Data
d <- 2

mat_diff <- abs(outer(0:(d - 1), 0:(d - 1), "-"))
sigma <- 0.99 ^ mat_diff # highly correlated

# Get the Cholesky factor
sigma_chol <- t(chol(sigma))

# Write model to file
stan_file <- "./stan/multi_normal.stan"
writeLines(normal_code, stan_file)

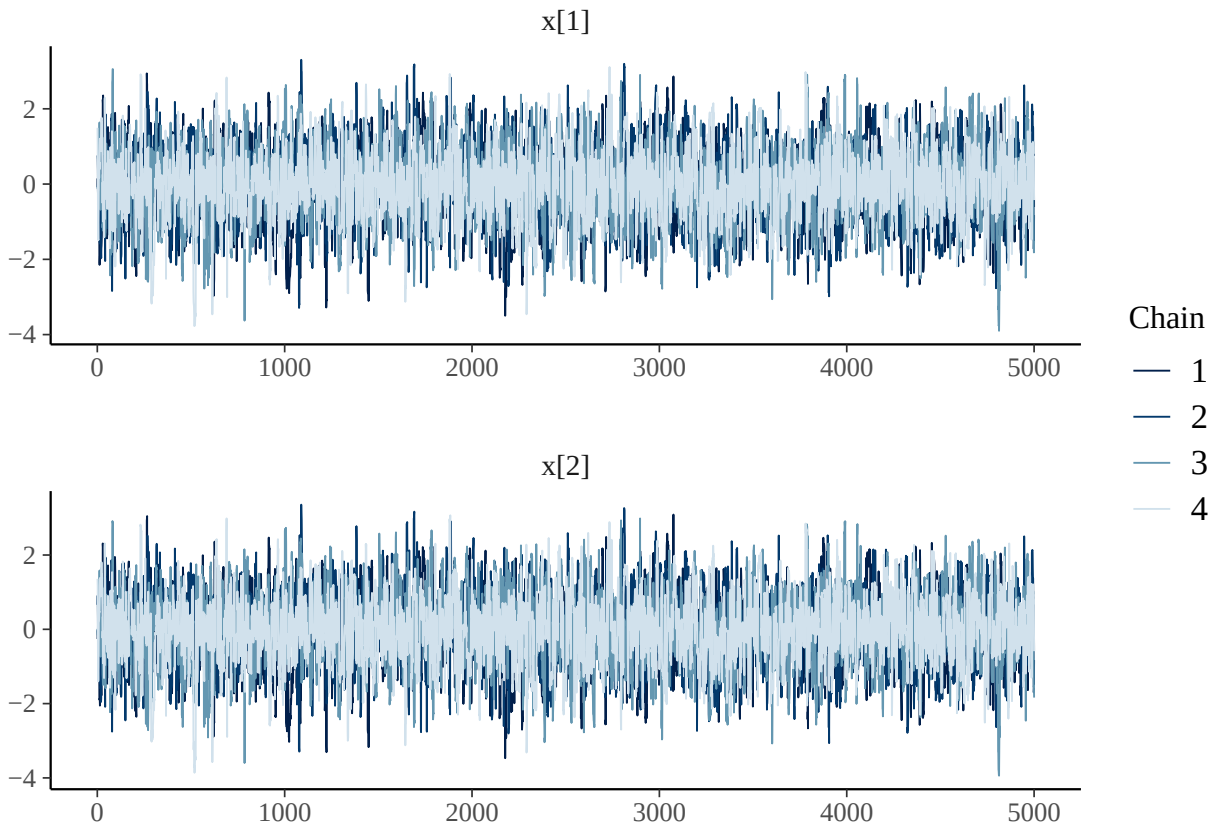
# Compile stan model
stan_model <- cmdstan_model(stan_file)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 1 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 1 Iteration: 2000 / 6000 [ 33%]  (Sampling)
```

```
## Chain 2 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 2 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 3 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 3 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 4 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 4 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 4 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 3 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 4 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 1 finished in 0.4 seconds.
## Chain 3 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 3 finished in 0.4 seconds.
## Chain 2 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 4 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 2 finished in 0.4 seconds.
## Chain 4 finished in 0.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 0.4 seconds.
## Total execution time: 0.7 seconds.
```

```
## # A tibble: 3 x 10
##   variable    mean   median    sd   mad    q5     q95  rhat ess_bulk ess_tail
##   <chr>      <dbl>    <dbl> <dbl> <dbl> <dbl>   <dbl> <dbl>    <dbl>    <dbl>
## 1 lp__     -0.997  -0.697   0.996 0.715 -3.00 -0.0482  1.00    5126.    6427.
## 2 x[1]     -0.0215 -0.00876 0.993 0.983 -1.67  1.59    1.00    3704.    3770.
## 3 x[2]     -0.0208 -0.00706 0.992 0.979 -1.66  1.60    1.00    3723.    3673.
```

```
mcmc_trace(draws_df, facet_args = list(ncol = 1), pars = c("x[1]", "x[2]")) # Stacked vertically, for es
```



Let's see how **stan** has explored the level set of the target distribution:

```
# Make sure you have the mvtnorm package installed for multivariate normal density:
# install.packages("mvtnorm")
library(mvtnorm)
```

```
## Warning: il pacchetto 'mvtnorm' è stato creato con R versione 4.4.3
```

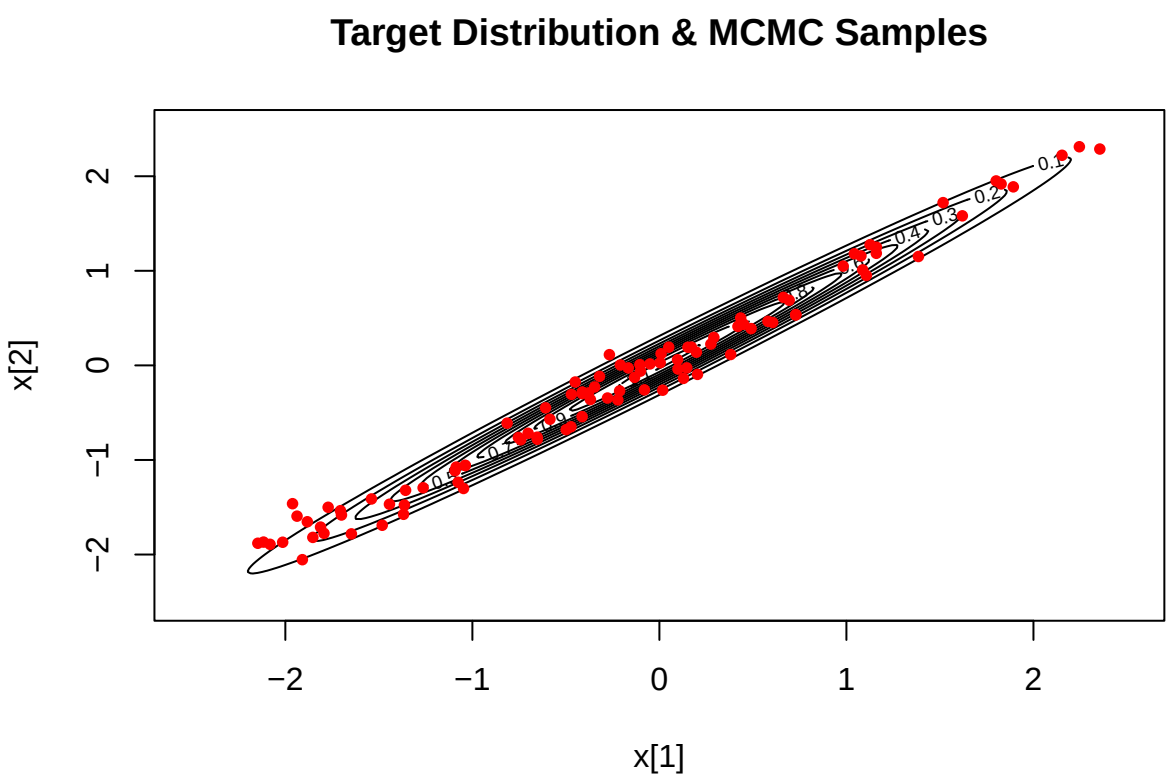
```
x_vals <- seq(-2.5, 2.5, length.out = 200)
grid_df <- expand.grid(x1 = x_vals, x2 = x_vals)

# Evaluate the multivariate normal probability density function
z_vals <- dmvnorm(grid_df, mean = c(0, 0), sigma = sigma)
z_matrix <- matrix(z_vals, nrow = length(x_vals), ncol = length(x_vals))

N <- 100
# Extract draws as a 3D array: iterations X chains X variables
draws_arr <- stan_fit$draws(variables = "x", format = "array")


x_samples <- draws_arr[1:N,1,1:2]
x_points <- x_samples[, , 1]
y_points <- x_samples[, , 2]
contour(x_vals, x_vals, z_matrix,
        xlab = "x[1]", ylab = "x[2]",
```

```
        main = "Target Distribution & MCMC Samples")
points(x_points, y_points, col = "red", pch = 16, cex = 0.8)
```



Compute running average of $\sum x_i$ when $d = 10$

```
d <- 10
mat_diff <- abs(outer(0:(d - 1), 0:(d - 1), "-"))
sigma <- 0.8 ^ mat_diff
sigma_chol <- t(chol(sigma))

normal_data <- list(
  dim = d,
  cov_chol = sigma_chol
)

stan_fit <- stan_model$sample(
  data = normal_data,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 5000,
  refresh = 1000,
  show_messages = F
)
```

```r
# Extract draws as a 3D array: [iterations, chains, variables]
draws_arr <- stan_fit$draws(variables = "x", format = "array")

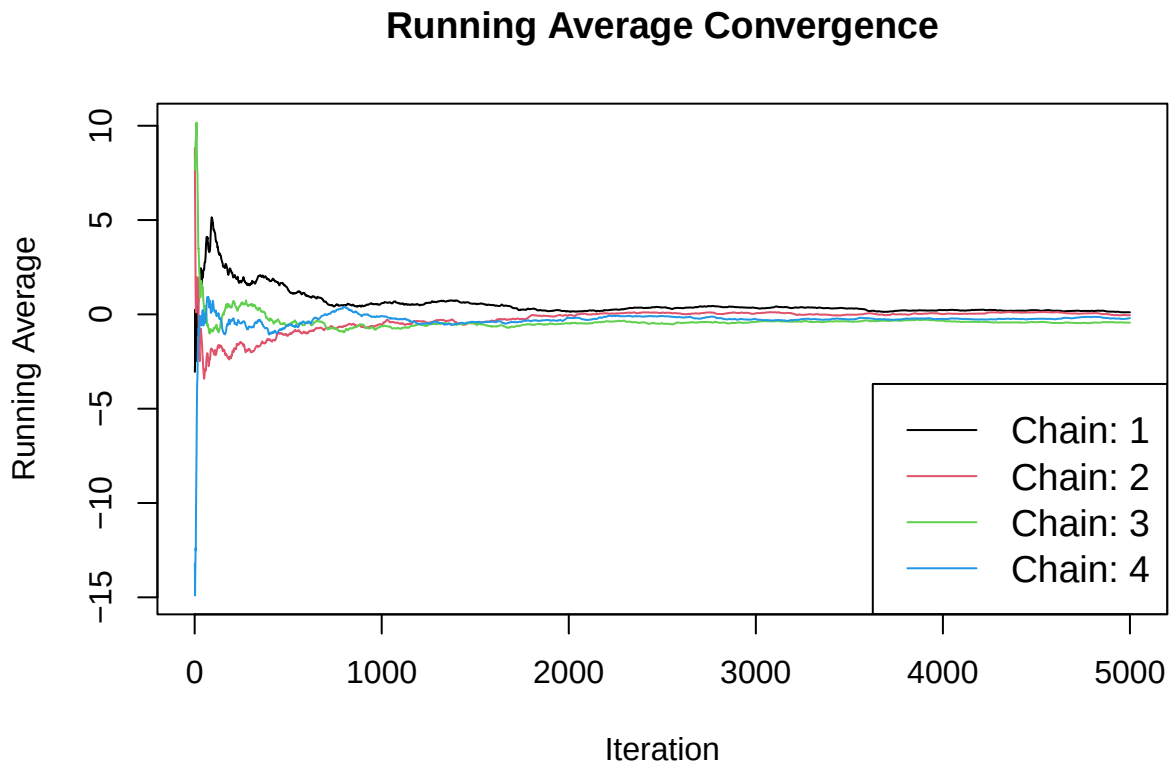
sum_x <- apply(draws_arr, c(1, 2), sum) # Results in a matrix of [iterations X chains]

n_iter <- nrow(sum_x)
running_sum <- apply(sum_x, 2, cumsum)
running_avg <- running_sum / (1:n_iter)

plot(1:n_iter, running_avg[, 1], type = "l", col = 1,
     ylim = range(running_avg),
     xlab = "Iteration", ylab = "Running Average",
     main = "Running Average Convergence")

# Add lines for the remaining chains
for (i in 2:ncol(running_avg)) {
  lines(1:n_iter, running_avg[, i], col = i)
}

# Add a legend
legend("bottomright",
       legend = paste("Chain:", 1:ncol(running_avg)),
       col = 1:ncol(running_avg),
       lty = 1,
       cex = 1.2)
```



Multiple chains are useful for assessing convergence - $\hat{R}$ and ESS

$\hat{R}$ is a diagnostic of convergence: Implementation in Stan follows Vehtari et al. (2021). Check it out for further details.

In the equations below, N is the number of draws per chain, M is the number of chains, $S = MN$ is the total number of draws from all chains, $\theta^{(nm)}$ is n-th draw of m-th chain, $\bar{\theta}^{(\bullet m)}$ is the average of draws from m-th chain, and $\bar{\theta}^{(\bullet\bullet)}$ is average of all draws.

1. For each scalar quantity of interest θ, first compute B and W, the between- and within-chain variances:

$$B = \frac{N}{M-1}\sum_{m=1}^{M}\left(\bar{\theta}^{(\bullet m)} - \bar{\theta}^{(\bullet\bullet)}\right)^2$$

$$W = \frac{1}{M}\sum_{m=1}^{M} s_m^2 \quad \text{where} \quad s_m^2 = \frac{1}{N-1}\sum_{n=1}^{N}\left(\theta^{(nm)} - \bar{\theta}^{(\bullet m)}\right)^2$$

2. Estimate $\text{var}(\theta \mid y)$, the marginal posterior variance of the estimand, by a weighted average of W and B:

$$\hat{\text{var}}^{+}(\theta \mid y) = \frac{N-1}{N}W + \frac{1}{N}B$$

3. Compute $\hat{R}$ as:

$$\hat{R} = \sqrt{\frac{\hat{\text{var}}^{+}(\theta \mid y)}{W}}$$

In practice, we monitor convergence of the iterative simulations to the target distribution by estimating the factor by which the scale of the current distribution for θ might be reduced if the simulations were continued in the limit $N \to \infty$. In fact, the above estimator, for an ergodic process, declines to 1 as $N \to \infty$. Hence, a value as close as possible to 1 can indicate that the chain converged. **Rule of thumb**: if $\hat{R} > 1.1$, the chain did not converge!

> *Indeed the actual implementation of* $\hat{R}$ in Stan does not use actual draws $\theta^{(nm)}$'s, but either their rank normalization or the rank normalization of the folded draws, i.e. the absolute deviation of $\theta^{(nm)}$ from the median. See Vehtari et al. (2021) for all the details.

The **effective sample size** (ESS) is an estimate of the sample size required to achieve the same level of precision if that sample was a simple random sample. Still, refer to Section 3.2 in Vehtari et al. (2021) for the estimation of ESS with multiple chains. We want it to be as close as possible to the actual draws of our chain. Notice that with Stan, if we are close to normality, we can reach values that are higher than the sample size in case of parameters whose posterior is close to a Gaussian distribution and low dependence on other parameters. Long story short, this happens due to the fact that HMC can exhibt autocorrelation plots with negative autocorrelations on odd lags. In the paper, you can find more details about the computation of the so-called *bulk-ESS* and *tail-ESS*, both available in Stan.

```
stan_fit$summary()
```

```
## # A tibble: 11 x 10
##    variable     mean   median    sd   mad    q5   q95  rhat ess_bulk ess_tail
##    <chr>       <dbl>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>    <dbl>
##  1 lp__     -4.99    -4.66    2.22  2.10  -9.13 -1.99  1.00    8089.   13176.
##  2 x[1]     -0.00371 -0.00273 0.996 0.994 -1.62  1.64  1.00    7829.   11067.
```

```
##  3 x[2]     -0.00692 -0.00992 1.00  1.01  -1.64  1.65  1.00    6844.   10229.
##  4 x[3]     -0.0145  -0.0195  0.999 1.01  -1.64  1.65  1.00    6272.    8979.
##  5 x[4]     -0.0162  -0.0194  0.998 1.00  -1.66  1.64  1.00    6033.    9011.
##  6 x[5]     -0.0176  -0.0189  1.00  0.990 -1.67  1.63  1.00    5921.    9222.
##  7 x[6]     -0.0202  -0.0163  0.997 0.997 -1.67  1.61  1.00    6192.    9361.
##  8 x[7]     -0.0195  -0.0107  0.994 0.998 -1.66  1.61  1.00    6163.    9876.
##  9 x[8]     -0.0192  -0.0165  0.998 0.991 -1.67  1.63  1.00    6519.   10271.
## 10 x[9]     -0.0139  -0.00520 0.992 0.985 -1.66  1.62  1.00    6961.   10172.
## 11 x[10]    -0.00845 -0.00793 0.997 0.989 -1.67  1.64  1.00    8404.   12687.
```

```
draws_x <- stan_fit$draws("x")
posterior::ess_bulk(draws_x)
```

```
## [1] 65706.22
```

```
posterior::ess_tail(draws_x)
```

```
## [1] 100712.8
```

```
posterior::rhat(draws_x)
```

```
## [1] 1.000015
```

Using stan to sample from Bayesian models

Just as before, but now the covariance is random, while x is observed

Model

$$\begin{aligned} x_1, \ldots, x_N \mid \Sigma &\sim \mathcal{N}(0, \Sigma) \\ \Sigma &\sim \mathcal{IW}(\nu_0, \psi_0) \end{aligned}$$

```
normal_unk_cov_code <- "
    data {
        int<lower=0> dim;
        int<lower=0> N;
        array[N] vector[dim] x;

        real<lower=0> nu0;
        cov_matrix[dim] psi0;
    }

    parameters {
        cov_matrix[dim] sigma;
    }

    transformed parameters {
        matrix[dim, dim] sigma_chol = cholesky_decompose(sigma);
    }
```

```
    model {
        vector[dim] mu = rep_vector(0, dim);
        sigma ~ inv_wishart(nu0, psi0);
        x ~ multi_normal_cholesky(mu, sigma_chol);
    }
"

# Generate data
d <- 10
N <- 500
mat_diff <- abs(outer(0:(d - 1), 0:(d - 1), "-"))
sigma <- 0.8 ^ mat_diff
sigma_chol <- t(chol(sigma))
x <- matrix(rnorm(N * d), nrow = N) %*% t(sigma_chol)

# Write model to file
stan_file <- "./stan/multi_normal_unk_cov.stan"
writeLines(normal_unk_cov_code, stan_file)

# Compile stan model
stan_model <- cmdstan_model(stan_file, quiet = TRUE)
```

```
normal_data <- list(
    dim = d,
    N = nrow(x),
    nu0 = 15,
    x = x,
    psi0 = (15 - d - 1) * diag(d) # centered at diag(d)
)

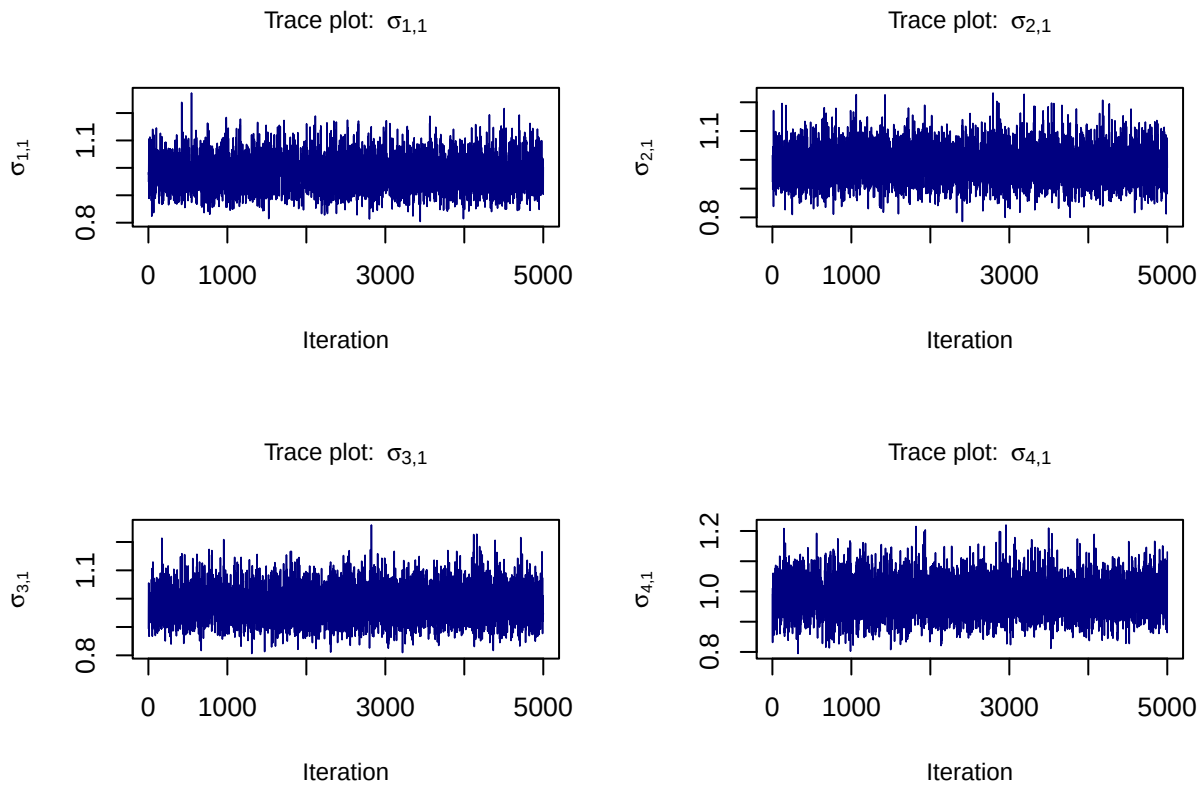
stan_fit <- stan_model$sample(data = normal_data, chains = 4, parallel_chains = 4,
                              iter_warmup = 1000, iter_sampling = 5000, show_messages = F, refresh = 10
```

```
stan_fit
```

```
##    variable    mean  median   sd  mad      q5     q95 rhat ess_bulk ess_tail
##  lp__       -133.46 -133.15 5.33 5.35 -142.70 -125.28 1.00     7873    13017
##  sigma[1,1]    0.98    0.98 0.06 0.06    0.89    1.09 1.00    18552    16345
##  sigma[2,1]    0.82    0.81 0.06 0.06    0.73    0.92 1.00    15719    15367
##  sigma[3,1]    0.64    0.64 0.05 0.05    0.56    0.73 1.00    13924    14223
##  sigma[4,1]    0.53    0.53 0.05 0.05    0.45    0.62 1.00    12749    13178
##  sigma[5,1]    0.46    0.46 0.05 0.05    0.39    0.55 1.00    11910    13835
##  sigma[6,1]    0.37    0.37 0.05 0.05    0.30    0.45 1.00    11445    13200
##  sigma[7,1]    0.32    0.31 0.05 0.05    0.24    0.39 1.00    12147    13864
##  sigma[8,1]    0.26    0.26 0.04 0.04    0.19    0.33 1.00    12727    14249
##  sigma[9,1]    0.17    0.17 0.04 0.04    0.09    0.24 1.00    14589    14928
##
##  # showing 10 of 201 rows (change via 'max_rows' argument or 'cmdstanr_max_rows' option)
```

```
draws_arr <- stan_fit$draws(variables = "sigma", format = "array")
par(mfrow = c(2,2))
for (i in 1:4) {
```

```
  plot(
    draws_arr[, i, 1],
    type = "l",
    col = "navyblue",
    lwd = 0.8,
    xlab = "Iteration",
    ylab = bquote(sigma[.(i)*",1"]),
    main = bquote("Trace plot: " ~ sigma[.(i)*",1"]),
    cex.main = 0.95,
    cex.lab = 0.9
  )
}
```




```
par(mfrow = c(1,1))
```

Going hierarchical

We consider 341 frogs from 12 genuses. For each frog we measure a function of the Body Weight (BoW) and the Brain Weight (BrW):

$$y_j = \log\left(\frac{BrW/BoW}{1 - BrW/BoW}\right) = \text{logit}(BrW/BoW), \quad j = 1, \ldots, 341$$

We want to study differences across the genuses

Denote with $g(j) \in \{1, \ldots, 12\}$ the genus of individual j. A simple hierarchical model is

$$\begin{aligned} y_j \mid \theta_1, \ldots, \theta_{12}, \sigma &\sim \mathcal{N}(\theta_{g(j)}, \sigma^2) \\ \theta_1, \ldots, \theta_{12} \mid \mu, \tau &\sim \mathcal{N}(\mu, \tau^2) \\ \mu \mid \tau &\sim \mathcal{N}(\mu_0, s_0^2) \\ \tau &\sim \text{Uniform}(0, \tau_{\max}) \\ \sigma &\sim \text{Uniform}(0, \sigma_{\max}) \end{aligned}$$

How to set the prior?

The ratio between bodyweight and brainweight is well studied and also has a Wikipedia page. Since we are not biologist, we use this page to elicit our prior.

```
invlogit <- function(x) {
    return(exp(x) / (1 + exp(x)))
}

logit <- function(x) {
    return(log(x) - log(1 - x))
}

# Average brain to body ratio for frogs is 1/172
mu0 <- logit(1 / 172)
cat(sprintf("mu0: %s\n", mu0))
```

```
## mu0: -5.14166355650266
```

```
# P(a <= X <= b), with X ~ N(mu,s)
p_in_interval <- function(mu, s, a, b) {
    return(pnorm(b, mean = mu, sd = s) - pnorm(a, mean = mu, sd = s))
}

# Compute s0 s.t. P(a <= X <= b) >= 0.9
loss_fn <- function(x) {
    return((p_in_interval(mu0, x, logit(1.0 / 550), logit(1.0/125)) - 0.9)^2)
}
s0 <- optim(10, loss_fn, method = "BFGS")$par
cat(sprintf("s0: %s\n", s0))
```

```
## s0: 8.90800222380683
```

```
# Raughly estimate the standard deviations of the ratios to estimate taumax
ratios <- 1.0 / c(12.0, 40.0, 40, 100, 125, 172, 550, 560, 600, 2496, 2798)
logit_ratios <- logit(ratios)
taumax <- 5 * sd(logit_ratios) * sqrt((length(logit_ratios) - 1) / length(logit_ratios))
cat(sprintf("taumax: %s\n", taumax))
```

```
## taumax: 8.40128217111209
```

```r
# No idea on sigmamax: set to 100
sigmamax <- 100.0
cat(sprintf("sigmamax: %s\n", sigmamax))
```

```
## sigmamax: 100
```

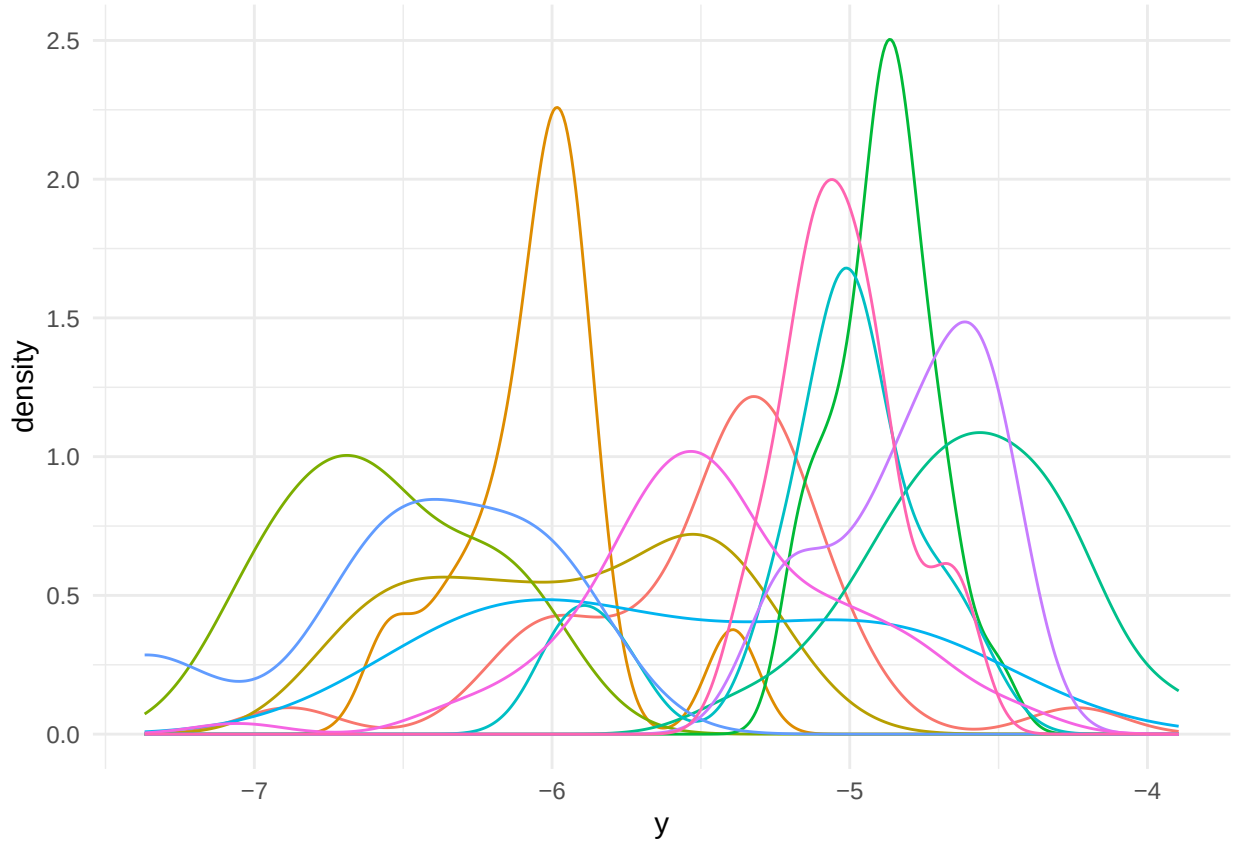
Let's import the data

```r
frog_data <- read.csv("frog_data.csv")
head(frog_data)
```

```
##   Order         Family   Genus Species     BoW   SVL   BrW
## 1 Anura Bombinatoridae Bombina  maxima 18543.9 60.24 45.30
## 2 Anura Bombinatoridae Bombina  maxima 11701.3 61.54 52.98
## 3 Anura Bombinatoridae Bombina  maxima 22450.8 59.80 58.70
## 4 Anura Bombinatoridae Bombina  maxima 23549.8 60.39 33.80
## 5 Anura Bombinatoridae Bombina  maxima 23091.0 60.51 40.00
## 6 Anura Bombinatoridae Bombina  maxima 20670.2 61.89 41.00
```

```r
frog_y <- logit(frog_data$BrW / frog_data$BoW)
unique_genera <- unique(frog_data$Genus)
ind2genus <- match(frog_data$Genus, unique_genera)

ggplot(data.frame(y = frog_y, genus = frog_data$Genus), aes(x = y, color = genus)) +
  geom_density() +
  theme_minimal() +
  theme(legend.position = "none")
```




```
frogs_code <- "
data {
    int<lower=0> J;
    int<lower=0> N;
    array[N] real y;
    array[N] int<lower=1, upper=J> g;

    real<lower=0> s0;
    real<lower=0> taumax;
    real mu0;
}

parameters {
    real mu;
    real<lower=0> tau;

    array[J] real theta;
    real<lower=0> sigma;
}


model {
    mu ~ normal(mu0, s0);
    tau ~ uniform(0, taumax);
    sigma ~ uniform(0, 100);
    theta ~ normal(mu, tau);
```

```
    for (i in 1:N) {
        y ~ normal(theta[g[i]], sigma);
    }
}
"

stan_file <- "./stan/frogs_code.stan"
writeLines(frogs_code, stan_file)

stan_model <- cmdstan_model(stan_file)
```

```
# Prepare input list
frogs_data <- list(
    J = 12,
    N = length(frog_y),
    y = frog_y,
    g = ind2genus,
    s0 = s0,
    taumax = taumax,
    mu0 = mu0
)

# Sample
stan_fit <- stan_model$sample(data = frogs_data, show_messages = F, refresh = 1000)
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/gbelt/AppDa

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar

## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or mi

## Chain 1

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/gbelt/AppDa

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar

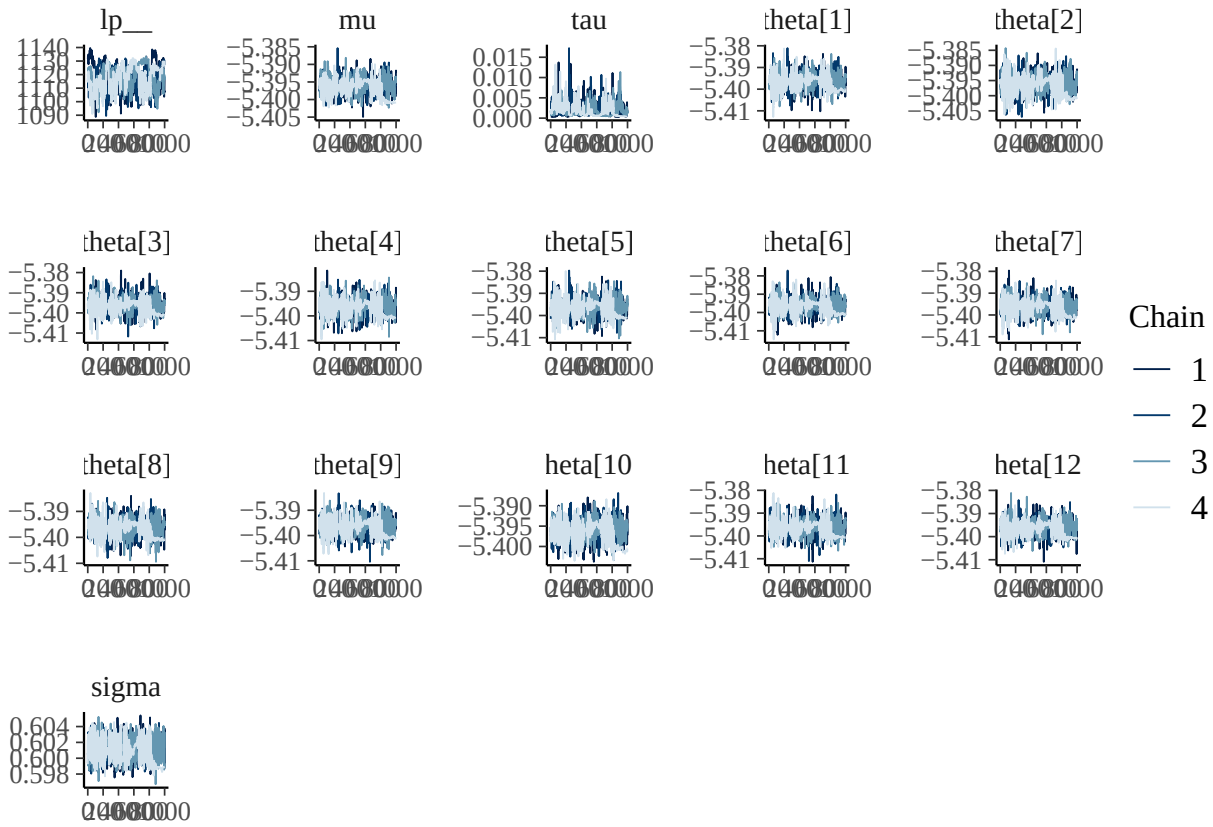
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or mi

## Chain 1

## Warning: 559 of 4000 (14.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.

## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

```
draws_arr <- stan_fit$draws(format = "array")
mcmc_trace(draws_arr, facet_args = list(ncol = 5))
```




```
num_div <- sum(stan_fit$sampler_diagnostics()[, , "divergent__"])
cat(sprintf("We have %d diverging iterations!\n", num_div))
```

```
## We have 559 diverging iterations!
```

"When the exact trajectories lie on energy level sets with regions of high curvature, the numerical trajectories generated by symplectic integrators can diverge, rapidly flying off towards the boundaries of phase space" (Betancourt 2018).

```
split_diverging <- function(stan_fit, var_name) {
    div_matrix <- stan_fit$sampler_diagnostics(format = "array")[, , "divergent__"] == 1
    vals <- stan_fit$draws(variables = var_name, format = "array")
    div_vals <- vals[div_matrix]
    non_div_vals <- vals[!div_matrix]
    return(list(div_vals, non_div_vals))
}
```

```
par(mfrow = c(1, 3))

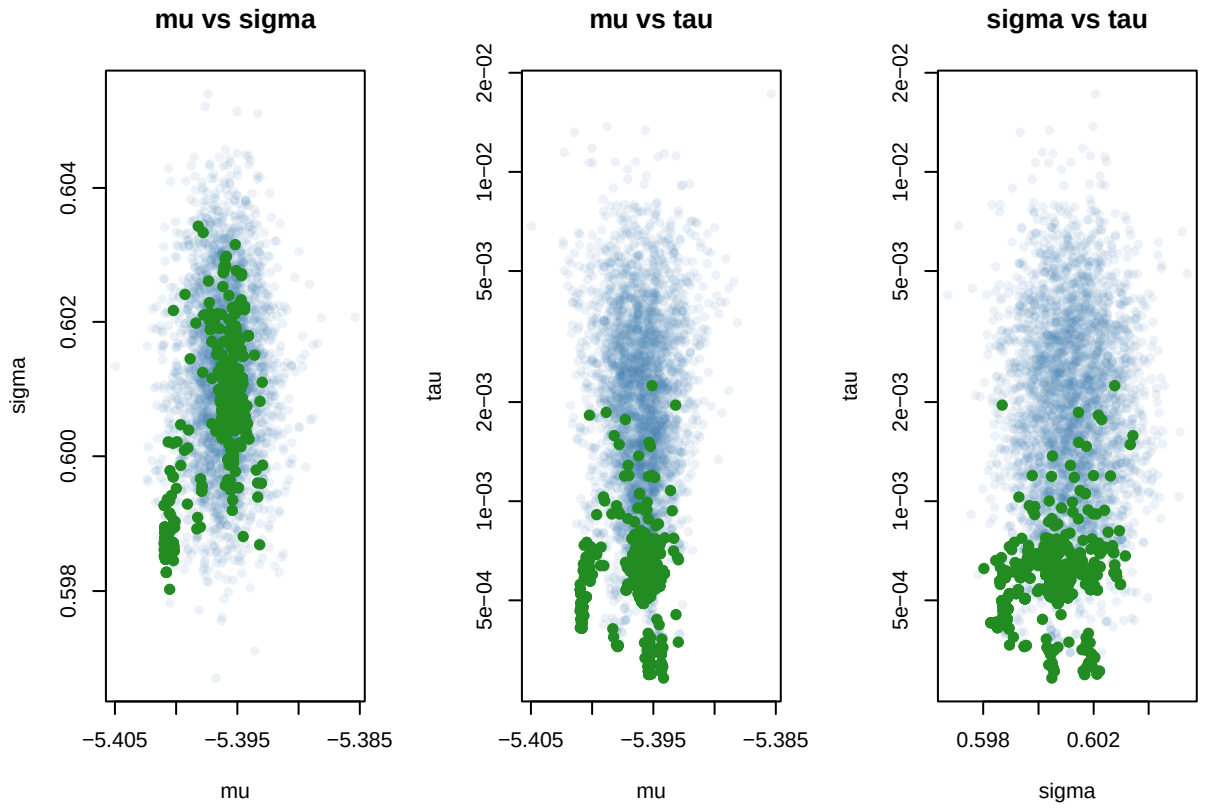
var_names <- c("mu", "sigma", "tau")
```

```r
for (i in 1:(length(var_names) - 1)) {
    v1 <- var_names[i]
    res1 <- split_diverging(stan_fit, v1)
    div1 <- res1[[1]]
    nondiv1 <- res1[[2]]

    for (j in (i + 1):length(var_names)) {
        v2 <- var_names[j]
        res2 <- split_diverging(stan_fit, v2)
        div2 <- res2[[1]]
        nondiv2 <- res2[[2]]

        log_x <- v1 %in% c("sigma", "tau")
        log_y <- v2 %in% c("sigma", "tau")
        log_arg <- paste0(ifelse(log_x, "x", ""), ifelse(log_y, "y", ""))

        plot(nondiv1, nondiv2, col = adjustcolor("steelblue", alpha.f = 0.1), pch = 16,
             log = log_arg, xlab = v1, ylab = v2, main = paste(v1, "vs", v2))
        points(div1, div2, col = "forestgreen", pch = 16, cex = 1.2)
    }
}
```




```r
par(mfrow = c(1, 1))
```

```
frogs_code_reparam <- "
data {
    int<lower=0> J;
    int<lower=0> N;
    array[N] real y;
    array[N] int<lower=1, upper=J> g;

    real<lower=0> s0;
    real<lower=0> taumax;
    real mu0;
}

parameters {
    real mu;
    real<lower=0> tau;

    vector[J] theta_raw;
    real<lower=0> sigma;
}

transformed parameters {
    vector[J] theta;
    theta = mu + theta_raw * tau;
}


model {
    mu ~ normal(mu0, s0);
    tau ~ uniform(0, taumax);
    sigma ~ uniform(0, 100);
    theta_raw ~ normal(0, 1);
    for (i in 1:N) {
        y ~ normal(theta[g[i]], sigma);
    }
}
"

stan_file <- "./stan/frogs_code_reparam.stan"
writeLines(frogs_code_reparam, stan_file)
stan_model <- cmdstan_model(stan_file)
```

```
## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
##   136 |   void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##       |        ^~~~~~~~~~
```

```
## C:/Users/gbelt/AppData/Local/Temp/RtmpYnMU8Y/model-1700588c523.hpp:522:3: note:   by 'void frogs_cod
##   522 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
```

```
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
##   205 |   void write_array(stan::rng_t& rng, std::vector<double>& theta,
##       |        ^~~~~~~~~~
```

```
## C:/Users/gbelt/AppData/Local/Temp/RtmpYnMU8Y/model-1700588c523.hpp:522:3: note:   by 'void frogs_cod
##   522 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
##    91 |   inline double log_prob(Eigen::VectorXd& theta,
##       |                 ^~~~~~~
## C:/Users/gbelt/AppData/Local/Temp/RtmpYnMU8Y/model-1700588c523.hpp:555:3: note:   by 'T_ frogs_code_
##   555 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr

## double>; std::ostream = std::basic_ostream<char>]' was hidden [-Woverloaded-virtual=]
##   159 |   inline math::var log_prob(std::vector<math::var>& theta,
##       |                    ^~~~~~~
## C:/Users/gbelt/AppData/Local/Temp/RtmpYnMU8Y/model-1700588c523.hpp:555:3: note:   by 'T_ frogs_code_
##   555 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crt
##    96 |   inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##       |                    ^~~~~~~
## C:/Users/gbelt/AppData/Local/Temp/RtmpYnMU8Y/model-1700588c523.hpp:555:3: note:   by 'T_ frogs_code_
##   555 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
##   154 |   inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##       |                 ^~~~~~~
## C:/Users/gbelt/AppData/Local/Temp/RtmpYnMU8Y/model-1700588c523.hpp:555:3: note:   by 'T_ frogs_code_
##   555 |   log_prob(Eig

## en::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
```

```
stan_fit <- stan_model$sample(data = frogs_data, refresh = 1000)
```

```
## Running MCMC with 4 sequential chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)

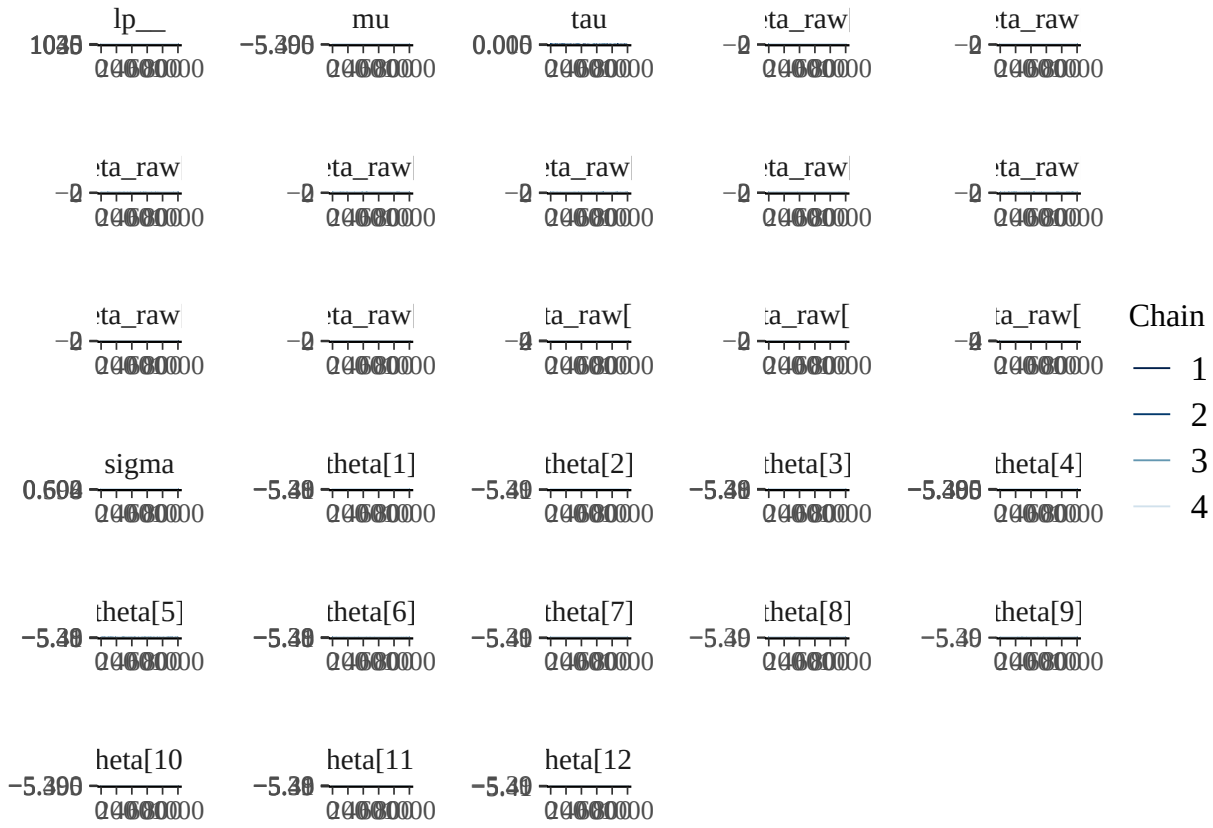
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 23.4 seconds.
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 20.5 seconds.
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 25.1 seconds.
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 48.5 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 29.4 seconds.
## Total execution time: 118.1 seconds.
```

```
sum(stan_fit$sampler_diagnostics()[, , "divergent__"])
```

```
## [1] 0
```

```
draws_arr <- stan_fit$draws(format = "array")
mcmc_trace(draws_arr, facet_args = list(ncol = 5))
```




```
par(mfrow = c(1, 3))

var_names <- c("mu", "sigma", "tau")

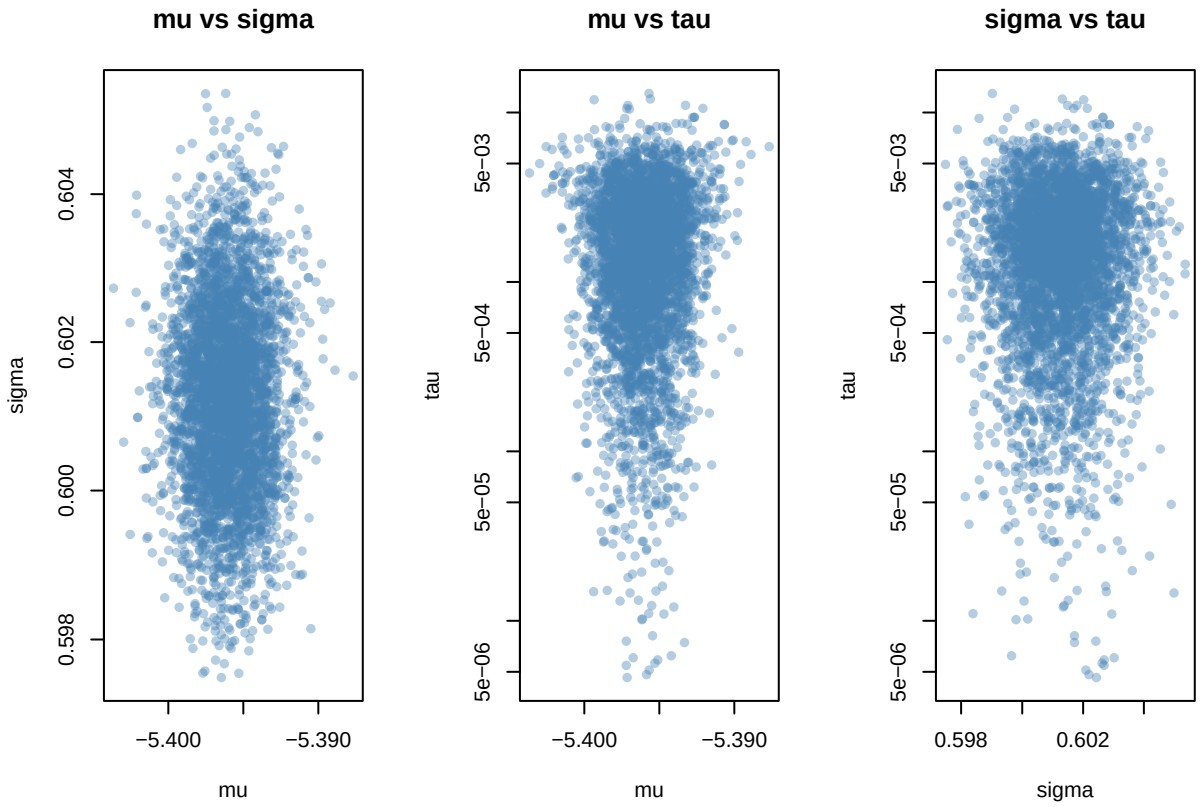
for (i in 1:(length(var_names) - 1)) {
    v1 <- var_names[i]
    res1 <- split_diverging(stan_fit, v1)
    div1 <- res1[[1]]
```

```
    nondiv1 <- res1[[2]]

    for (j in (i + 1):length(var_names)) {
        v2 <- var_names[j]
        res2 <- split_diverging(stan_fit, v2)
        div2 <- res2[[1]]
        nondiv2 <- res2[[2]]

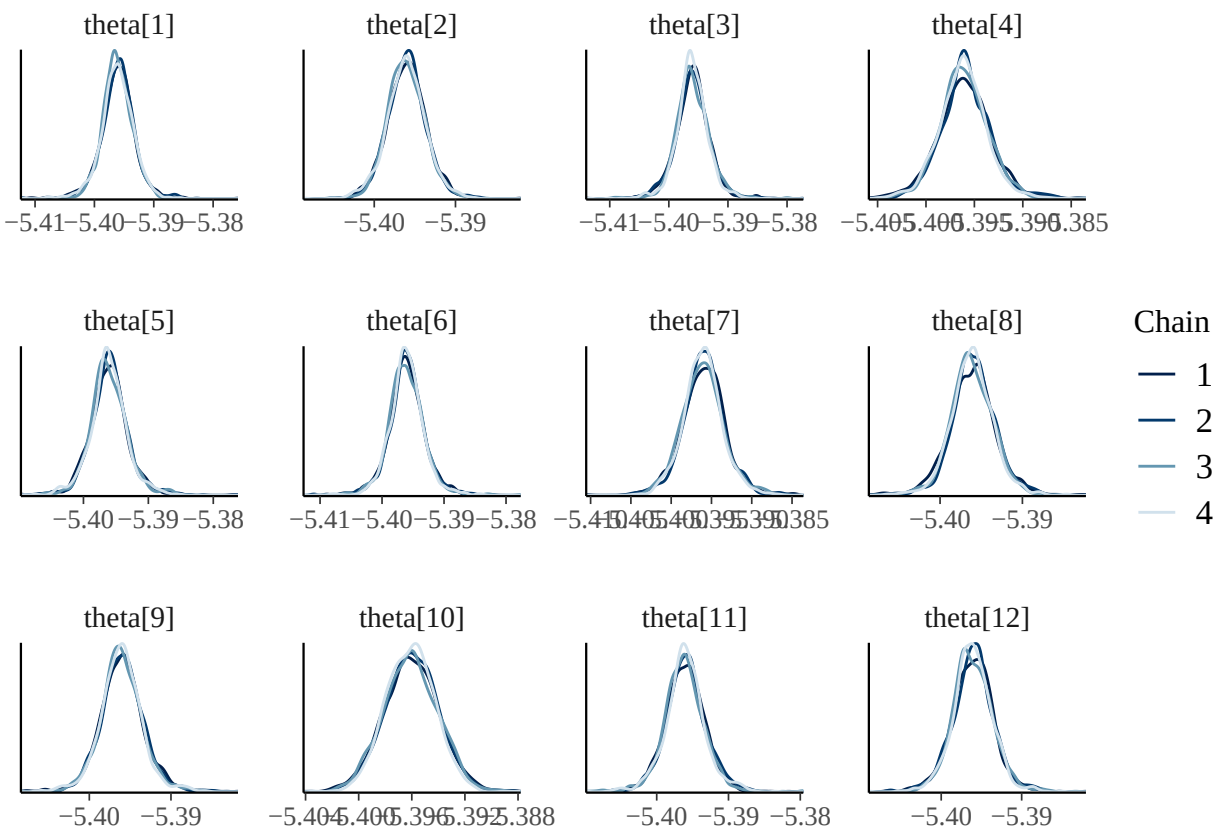
        log_x <- v1 %in% c("sigma", "tau")
        log_y <- v2 %in% c("sigma", "tau")
        log_arg <- paste0(ifelse(log_x, "x", ""), ifelse(log_y, "y", ""))

        plot(nondiv1, nondiv2, col = adjustcolor("steelblue", alpha.f = 0.4), pch = 16,
             log = log_arg, xlab = v1, ylab = v2, main = paste(v1, "vs", v2))
        points(div1, div2, col = "forestgreen", pch = 16, cex = 1.2)
    }
}
```




```
par(mfrow = c(1, 1))
```

```
mcmc_dens_overlay(stan_fit$draws(variables = "theta"))
```



Other startegies to fix diverging iterations

Increase `adapt_delta` (target acceptance rate) from the default value (0.8) to (0.9, 0.95, 0.99, 0.999)

Decrease the initial `step_size` from 1 (default value)

Increse `max_treedepth` (something very specific of NUTS) from 10

```
stan_fit <- stan_model$sample(
    data = frogs_data,
    adapt_delta = 0.99,
    step_size = 0.5,
    max_treedepth = 15
)
sum(stan_fit$sampler_diagnostics()[, , "divergent__"])
```

```
par(mfrow = c(1, 3))

var_names <- c("mu", "sigma", "tau")

for (i in 1:(length(var_names) - 1)) {
    v1 <- var_names[i]
```

```
    res1 <- split_diverging(stan_fit, v1)
    div1 <- res1[[1]]
    nondiv1 <- res1[[2]]

    for (j in (i + 1):length(var_names)) {
        v2 <- var_names[j]
        res2 <- split_diverging(stan_fit, v2)
        div2 <- res2[[1]]
        nondiv2 <- res2[[2]]

        log_x <- v1 %in% c("sigma", "tau")
        log_y <- v2 %in% c("sigma", "tau")
        log_arg <- paste0(ifelse(log_x, "x", ""), ifelse(log_y, "y", ""))

        plot(nondiv1, nondiv2, col = adjustcolor("steelblue", alpha.f = 0.4), pch = 16,
             log = log_arg, xlab = v1, ylab = v2, main = paste(v1, "vs", v2))
        points(div1, div2, col = "forestgreen", pch = 16, cex = 1.2)
    }
}

par(mfrow = c(1, 1))
```